zeroriscy_cs_registers

OVERVIEW

Control and Status Registers (CSRs) are special-purpose registers in the RISC-V processor used for controlling processor behavior, handling exceptions, managing interrupts, and storing important system-level information.
Unlike general-purpose registers, CSRs are not used for arithmetic or normal data computation. Instead, they act as the control panel of the CPU, managing system operations and maintaining processor state during execution.
In the Zero-Riscy core, the CSR block is responsible for implementing machine-level control and supporting the RISC-V privileged architecture. It provides the necessary infrastructure for interrupt handling, exception processing, performance monitoring, and system configuration.

WHAT IS A CSR?

A CSR (Control and Status Register) is a dedicated register used to:

- Control hardware operations
- Store processor status information
- Handle interrupts and exceptions
- Manage privilege and execution state

CSRs are essential for system-level functionality and processor supervision.
Important Note: CSRs are not used for arithmetic operations. Their primary purpose is control, monitoring, and exception management.

KEY FUNCTIONS: 1. CONTROL OPERATIONS

CSRs help configure and manage processor behavior. Examples include:

- Enabling or disabling interrupts
- Configuring CPU operating modes
- Managing privilege-level operations
- Handling debug and trap-related controls

This allows the processor to respond correctly to system events.

KEY FUNCTIONS: 2. STATUS INFORMATION

CSRs continuously store important processor state information such as:

- Interrupt status
- Exception cause
- Current processor mode
- Return addresses after exceptions
- Pending interrupt information

This helps the processor maintain correct execution flow and system reliability.

EXCEPTION AND INTERRUPT HANDLING

One of the most important roles of the CSR block is handling exceptions and interrupts. When an interrupt or exception occurs:

- The cause of the event is stored
- The current Program Counter (PC) is saved
- Control is transferred to the exception handler
- After handling, execution returns safely to the original program

This mechanism ensures reliable system operation even during unexpected events.

IMPORTANT CSRs IN RISC-V

Several machine-level CSRs are commonly used in Zero-Riscy.

- mstatus -> Stores machine status and control flags.

Used for global interrupt enable, privilege-related control, and processor execution state.

- mcause -> Stores the reason for an exception or interrupt.

Helps identify what caused the trap.

- mepc -> Stores the Program Counter of the interrupted instruction.

Used to return execution after exception handling.

- mtvec -> Stores the address of the trap handler.

This is where the processor jumps during interrupts or exceptions.

- mie -> Machine interrupt enable register. Controls which interrupts are allowed.
- mip -> Machine interrupt pending register. Indicates which interrupts are waiting to be serviced.

CSR INSTRUCTIONS

Special RISC-V instructions are used to access CSRs.

- CSRRW -> Read and Write CSR. Used to read the old value and write a new value.
- CSRRS -> Read and Set Bits. Used to set selected bits in a CSR.
- CSRRC -> Read and Clear Bits. Used to clear selected bits in a CSR.

Example: csrrw x1, mstatus, x2

This instruction writes the value of x2 into mstatus and stores the old value of mstatus into x1.

SIMPLE UNDERSTANDING & ARCHITECTURE RATIONALE

A simple way to understand CSR is: CSR = Control Panel of the CPU.

It is responsible for monitoring processor behavior, managing exceptions, handling interrupts, and maintaining execution control. It does NOT perform arithmetic calculations or normal ALU operations.

Why Zero-Riscy Uses CSR:

The Zero-Riscy processor uses CSRs because it follows the RISC-V privileged architecture standard. It specifically implements mstatus, mepc, and mcause. This allows compiler support, RTOS compatibility, standard interrupt handling, and reliable exception processing. Since Zero-Riscy supports only Machine Mode, the design remains simpler, smaller, and more efficient while still maintaining full functionality.

INPUT AND OUTPUT SIGNALS: BASIC SIGNALS

- clk, rst_n -> System clock and reset signals used for synchronous operation and initialization.
- core_id_i, cluster_id_i -> Processor identification signals.
- boot_addr_i -> Starting program address after reset.

INPUT AND OUTPUT SIGNALS: CSR INTERFACE

- csr_access_i -> Enables CSR access operation
- csr_addr_i -> Selects the target CSR register
- csr_wdata_i -> Input data for CSR write operations
- csr_op_i -> Defines the type of CSR operation (Read, Write, Set bits, Clear bits)
- csr_rdata_o -> Output data from the selected CSR returned to the processor.

INPUT AND OUTPUT SIGNALS: INTERRUPTS & PERFORMANCE

Interrupt and Exception Signals:

- m_irq_enable_o -> Global interrupt enable output
- mepc_o -> Return address after exception handling
- csr_save / csr_restore -> Used to save and restore processor state during traps

Performance Signals:

- if_valid_i, id_valid_i -> Used for instruction flow tracking
- imiss_i -> Indicates instruction memory miss
- jump_i, branch_i -> Used for control flow performance monitoring

FUNCTIONS OF CSR BLOCK & IMPLEMENTATION

Functions of CSR Block:

1. Status Tracking: Maintains processor state and system control information.
2. Trap Handling: Saves Program Counter and trap cause during interrupts and exceptions.
3. Performance Monitoring: Tracks processor activity using cycle counters and instruction counters.

Implementation Overview:

The CSR block uses a combination of:

- Combinational Logic -> Used for CSR address decoding and operation selection.
- Flip-Flops -> Used for storing CSR values and maintaining processor state.
- Multiplexers (MUX) -> Used for selecting the correct CSR output data.

CODE UNDERSTANDING: READ & WRITE LOGIC

Code Understanding (Structs):

In the RTL implementation, 'typedef struct' is used to group related control signals together (e.g., mie, mpp). This improves code readability and maintainability.

Read Operation:

Implemented using 'case(csr_addr_i)', which selects the required CSR based on its address.

Write Operation:

The selected CSR is updated with new values depending on operation type, write enable, and hardware priority.

Important Note: Hardware override always gets higher priority than software writes. This ensures correct exception and interrupt behavior.

PERFORMANCE COUNTERS & ASIC VS FPGA

Performance Counters:

- Cycle Counter: Increments continuously with every clock cycle. Used for measuring total execution time.
- Instruction Counter: Increments only when a valid instruction is executed. Used for instruction performance analysis.

These counters are useful for benchmarking and optimization.

ASIC vs FPGA (ifdef):

Different implementations are used depending on the target platform.

- ASIC: Fewer counters, reduced area usage, optimized for silicon efficiency.
- FPGA / Simulation: Full counters enabled, better visibility for testing and debugging.

FSM OF CSR (INTERRUPT HANDLING FLOW)

The CSR block follows a structured control flow mechanism for interrupt handling:

1. Interrupt occurs
2. Save PC to mepc
3. Store cause in mcause
4. Jump to handler using mtvec
5. Execute interrupt service routine
6. Restore state and resume execution

SUMMARY

The zeroriscy_cs_registers module is the central control and monitoring block of the Zero-Riscy processor. It handles interrupts and exceptions, stores machine-level status information, controls processor execution state, supports performance monitoring, and maintains RISC-V privileged architecture compliance. By managing system-level operations reliably, the CSR block ensures stable, secure, and efficient processor execution, making it one of the most important modules in the processor architecture.